2

Software Architecture

Architectural Styles

Includes content adapted from *Fundamentals of Software Architecture – An Engineering Approach*, 2020, O'Reilley

2.1

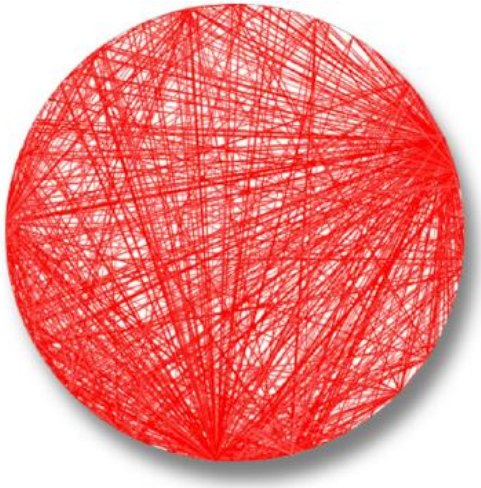
Introduction

Core concepts and the evolution of system architecture



Big Ball of Mud

- Refers to the *absence* of any discernible architecture structure.
  - E.g. a simple scripting application that grew too big for its own good.
- Hard to evolve, maintain, deploy, test, and generally ensure the support for any architectural characteristics.



Classes (on the perimeter) and dependencies on a Big Ball of Mud

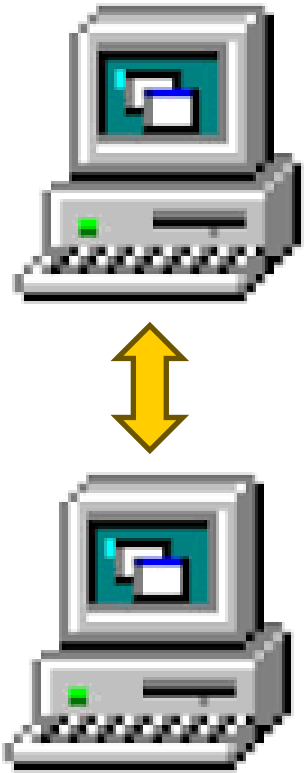
Unitary Architecture

- An architecture focused on a system based on a single machine.
- Nowadays, increasingly rare. Typically found only in:
  - Legacy systems
  - Embedded systems



Client/Server

- Separation of technical functionality between frontend and backend, a *two tier*, or client/server architecture.
- Manifested in different flavours over the years...
  - Desktop + database
  - Browser+web server
  - Three-tier architectures – Late 90s, database tier, Application server tier and fronted, using low-level protocols such as CORBA, DCOM.





Monolithic vs Distributed

- Two main types or classes of architectures.
- **Monolythic:**
  - Layered Architecture
  - Pipeline Architecture
  - Microkernel Architecture
- **Distributed:**
  - Service-based architecture
  - Event-driven architecture
  - Space-based architecture
  - Service-oriented architecture
  - Microservices architecture

2.2

Challenges in Distributed Architectures

Distributed pains...



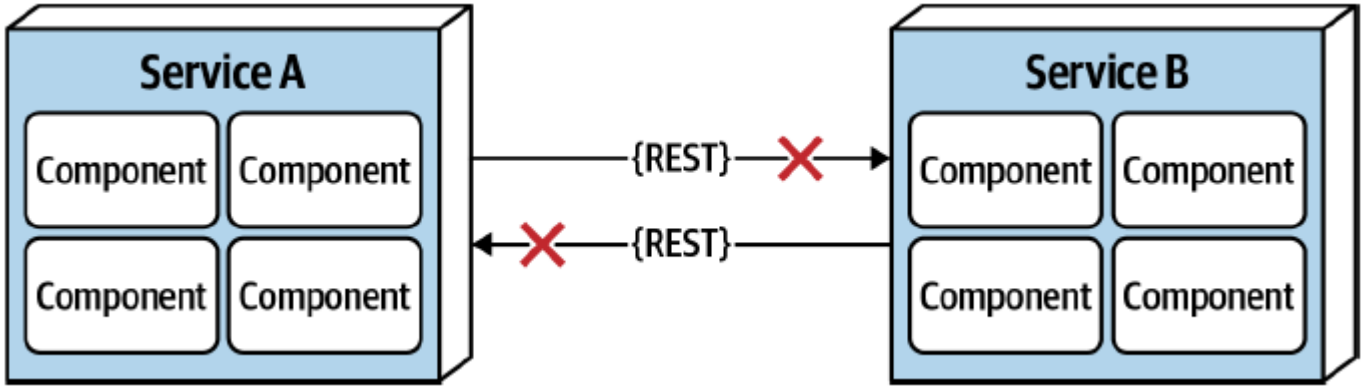
Monolithic vs Distributed

- Distributed architectures can offer higher performance, scalability and availability than their monolithic counterparts.
- However, everything is a trade-off.
- Distributed architectures share a common set of challenges and issues not found in monolithic architectures.
- We are going to examine:
  - Fallacies and flawed assumptions when creating distributed architectures
  - Specific challenges associated to distributed architectures

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*... (L.P. Deutsch, Sun Microsystems, 1994)

Fallacy #1 – The network is Reliable

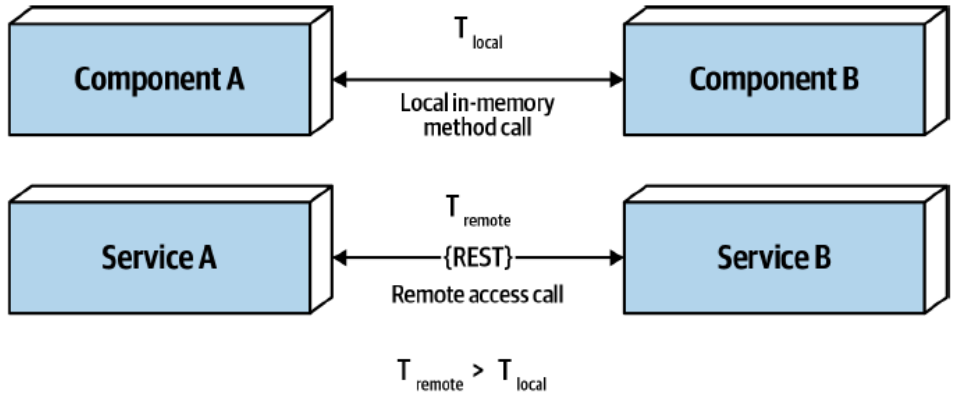


It is not. This impacts reliability and requires the use of techniques to address connectivity, such as circuit breakers.

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*...

Fallacy #2 – Latency is zero

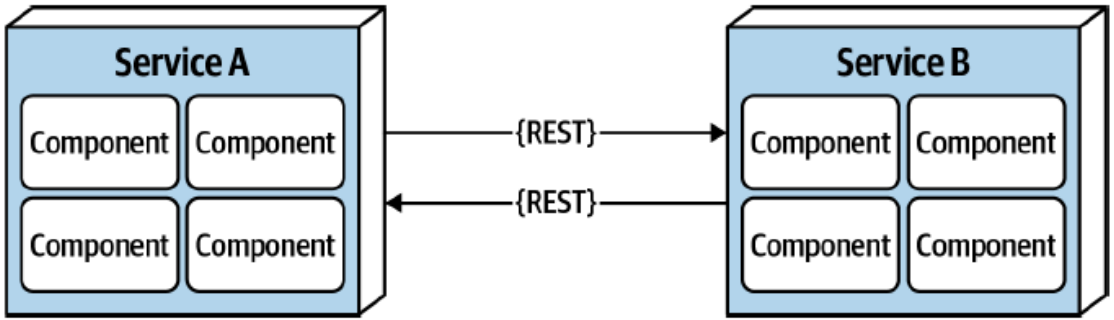


It is not. This compounds the cost of sequential service calls.
Know the latency average in your system, but also the 95th to 99th percentile…

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*...

Fallacy #3 – Bandwidth is infinite.

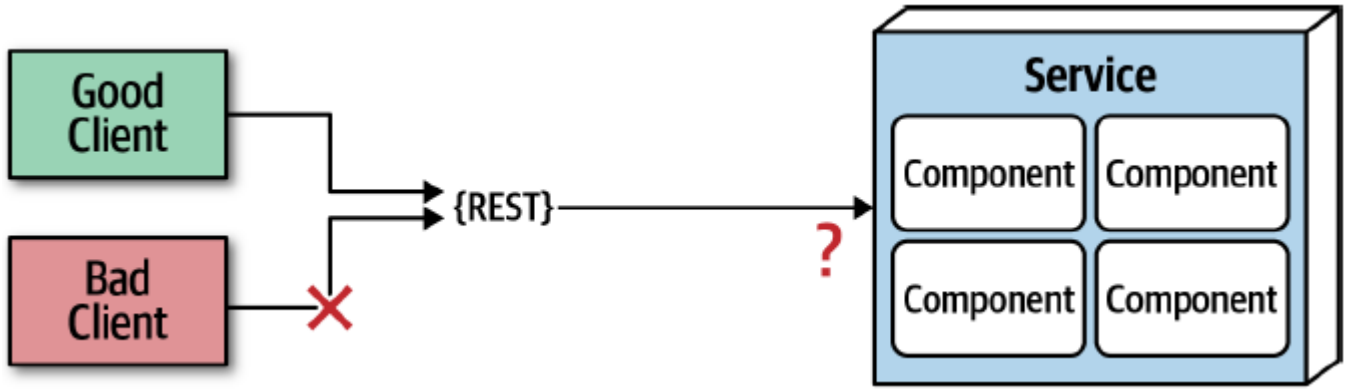


It is not. Avoid transmitting useless data (e.g. returning all user information when only the name is required). Remember that this inefficiency is multiplied by the number of service calls made…

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*...

Fallacy #4 – The network is secure

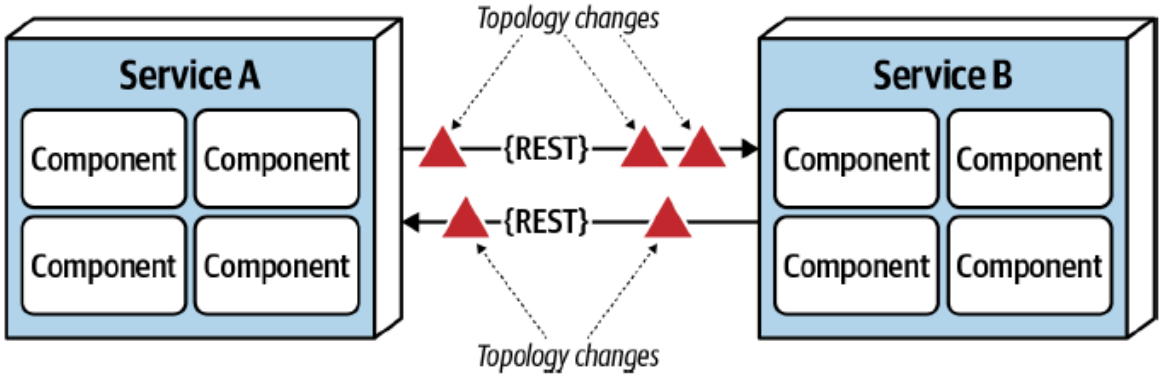


It is not. The surface of attack increases with each endpoint and component.

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*...

Fallacy #5 – The topology never changes



Of course it does… This can invalidate latency assumptions and trigger timeouts, circuit breakers, etc

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*...

Fallacy #6 – There is only one administrator...

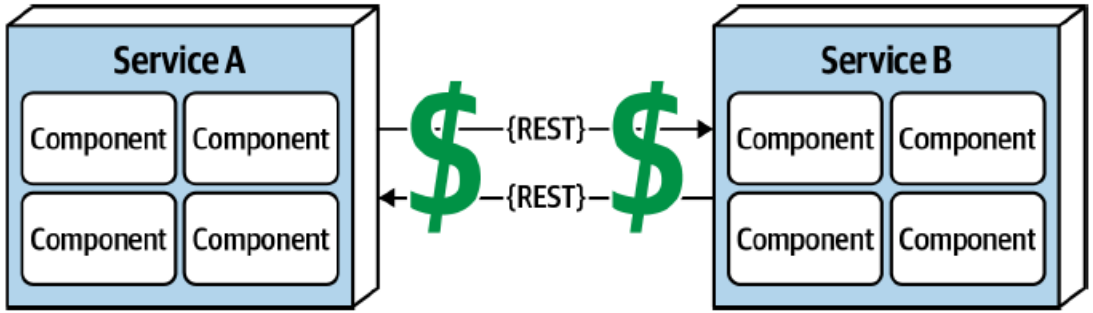


There are usually many admins on large organizations, and maintenance efforts are shared.

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*...

Fallacy #7 – Transport cost is zero...

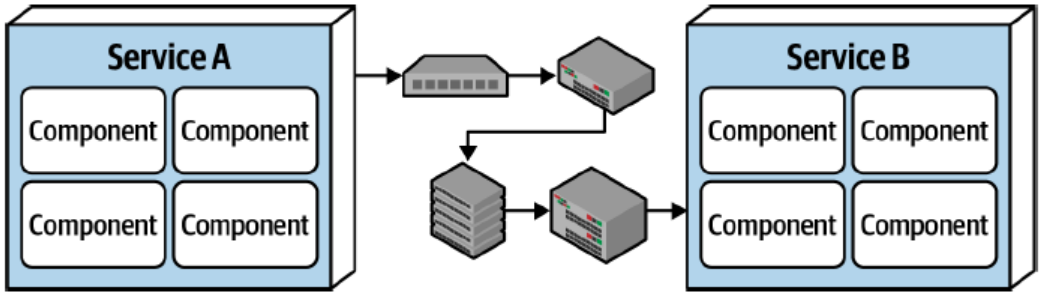


It is not… Carefully consider existing network resources before commiting…
What is the cost of making one API call?

Monolithic vs Distributed

- Distributed architectural styles offer higher performance, scalability, and availability. However, be aware of the *8 fallacies*...

Fallacy #8 – The network is homogenous...



It is not… Carefully consider existing network resources before committing, accounting for variable reliability, latency, bandwidth, etc



Other Challenges in Distributed Architectures

Some challenges are specifically tied to the distributed nature of systems are:

- **Distributted Logging –** There are typically dozens to hundreds of distributed logs, making it difficult to track down a problem to identify the root cause.
- **Distributed Transactions –** It is easy to ensure standard commits and rollbacks and achieve ACID properties in monolithic systems. Consistency and integrity are usually sacrificed in some way for the sake of distribution.
- **Maintenance and Versioning** – Decoupled services may evolve independently, and things may break...

2.3

Architectural Styles

Blueprints of architectural solutions



Architectural Styles

- Architectural Styles (or patterns), much like design patterns, describe common solutions to architectural decisions.
- They provide a common language that can be used to communicate and discuss alternative options without the necessity of describing many low-level abstractions.
- We will proceed to discuss some fundamental architectural styles.

2.4

Layered Architecture Style

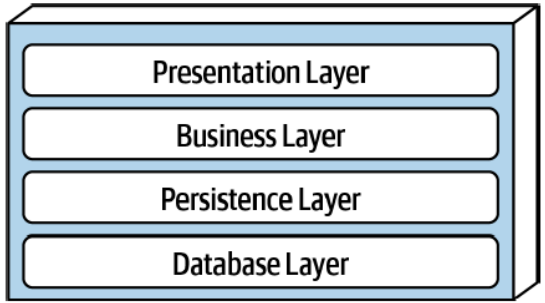
One of the more common Architectural Styles



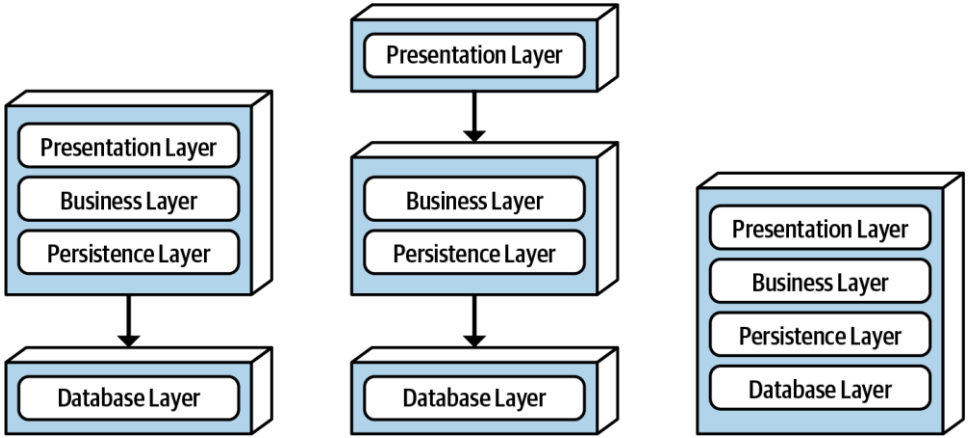
Layered Architecture

- Also known as *n-tiered* architecture style.
- *De facto* standard for most applications, due to simplicity, familiarity and low cost.
- Aligned with Conway's law:
  - User interface (UI) developers...
  - Backend developers...
  - Database experts...
- Often, the default choice when little consideration is provided to architectural concerns.
- Technically Partitioned Architecture
  - As a result, any particular business domain is spread throughout all layers.

Topology



- Logical View
  - Standard logical model
  - But other alternatives in terms of number and type of layers can be often be found.

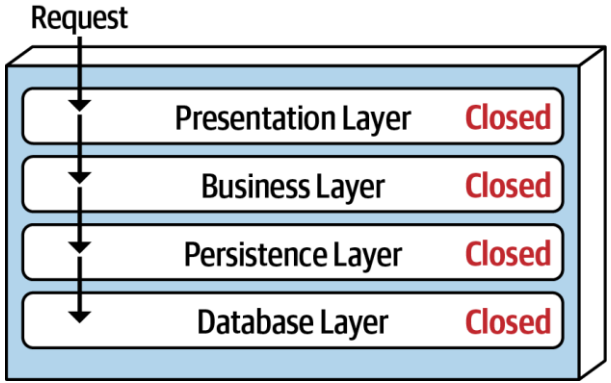


- Deployment or Physical View



Layer Isolation

- Each layer can be *closed* or *open*.
  - With closed layers, requests cannot skip any layer as they move top-down.
  - With open layers, requests can be made directly from the top layers to bottom layers, skipping intermediate layers.





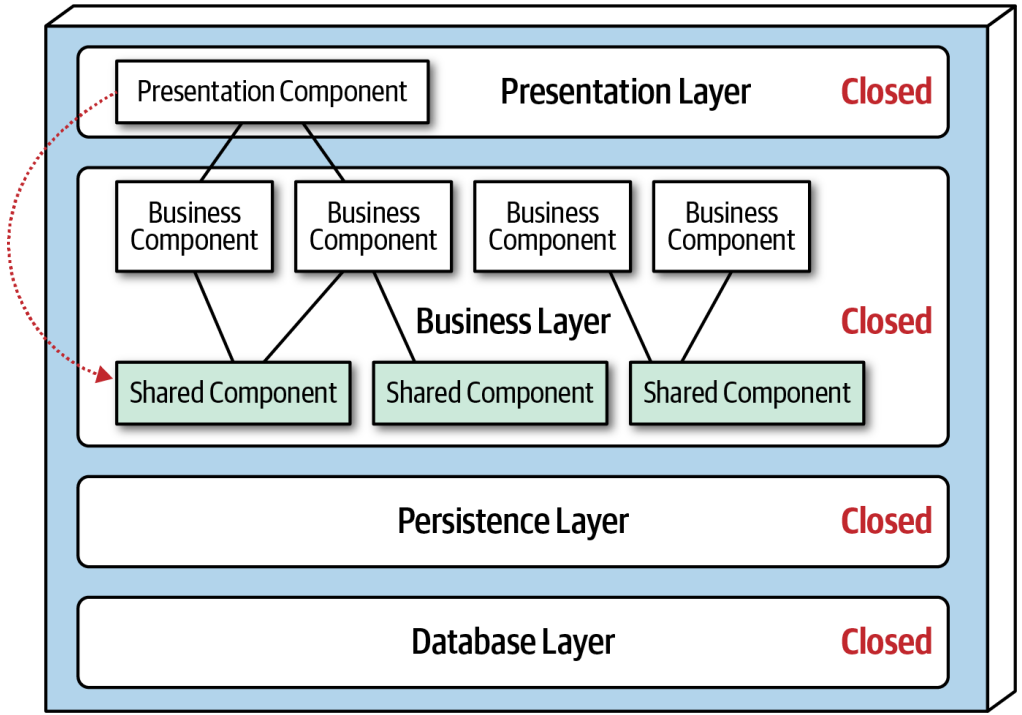
Layer Isolation

- *Layers of Isolation Concept:* Changes in one layer should not impact or affect components in other layers (provided inter layer contracts are maintained).
  - This is promoted by adoption of *closed layers*.
  - Layers can be replaced without impact.
- **Open Layers:** Simpler, faster, tighter coupling, brittle, difficult and expensive to change due to poor isolation.
- **Closed Layers:** Slower, cumbersome, looser coupling with better layer isolation.



Adding Layers

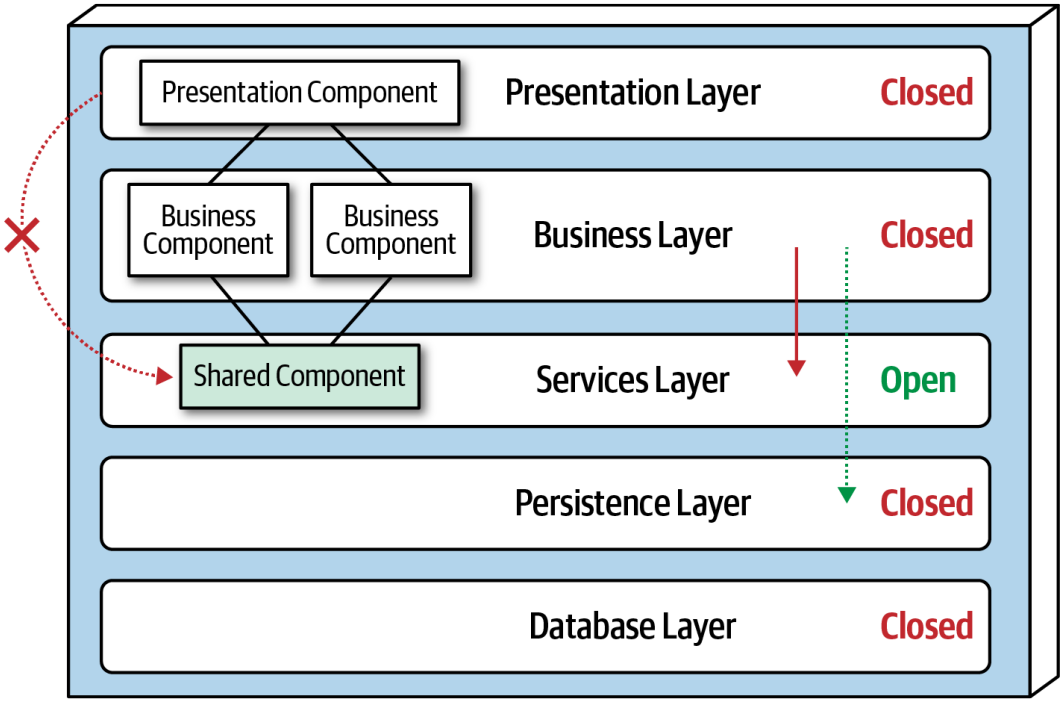
- In this scenario, a presentation component has architectural access to Shared Business component.
- This makes it harder to evolve and control the system.





Adding Layers

- One way to address this limitation is to add a new layer containing shared business objects.
- The shared component is no longer accessible to presentation.
- If this new layer is Open, then the Business layer will be able to access the Persistence layer as before.





Other Considerations

- *Architectural Sinkhole Antipattern* – Occurs when a significant number of requests are passed down and returned from layer to layer without additional processing.
  - 80-20 rule: You're good if only at most 20% of requests should fall into this pattern. If 80% of requests are sinkholes, find a new architecture! Otherwise, your judgement call.
  - Not a problem if all layers are open, but that comes with its own challenges...



When to Use?

- Good choice for small, simple applications or websites.
- Good starting point for scenarios with very tight budgets and time constraints.
  - Simplicity and familiarity makes it one of the lowest-cost options.
- Good starting point if business needs and requirements are still uncertain.
- Due to limited modularity, as applications grow in size and complexity, the attributes of maintainability, agility, deployability, and testability begin to suffer.



Architectural Characteristics Ratings

- ***Architectural Quanta*** – An independently deployable artifact with highly cohesive functionality and homogenous quality attributes.
- Monolithic nature avoids complexities associated with distributed architectures.
- Simple and easy understand, they are low cost to build and maintain.
  - *This advantage dissipates as system size increases.*

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★



Architectural Characteristics Ratings

- **Deployability** rates low. There is usually a lack of frequent deployments due to high deployment effort and high risk of complications due to single deployment unit.
- **Testability** is complicated by the monolithic structure, which hinders complete testing of small changes. Stubbing or mocking layers can help, however.
- **Reliability** is reasonable, due to the absence of issues due to network availability, bandwidth or latency. It suffers slightly due to poor testability.
- **Elasticity** and **Scalability** are very low due to monolithic deployment and lack of architectural modularity.
  - Not impossible, but limited and requires high effort.

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★



Architectural Characteristics Ratings

- **Performance** suffers from lack of parallel processing, closed layering and the architectural sinkhole antipattern.
- **Fault tolerance** is low, due to monolithic architecture. A single failure can bring the entire system down, and recovery time (startup time) can be high.

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★

2.4

Pipeline Architecture Style

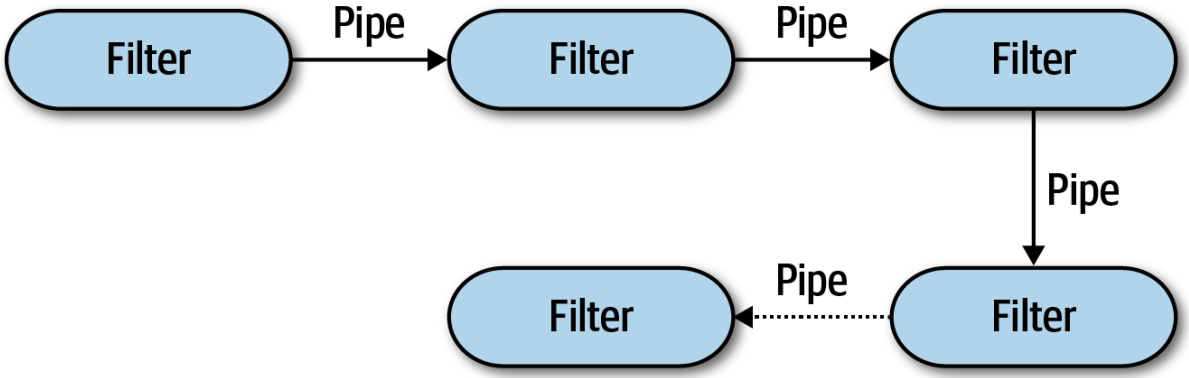
Splitting Functionality into Sequential Parts



Pipeline Architecture

- Also known as *pipes and filters,* architecture style.
- The underlying principle behind Unix Bash and Zsh.
- Functionality is decomposed in sequential steps, which will correspond to distinct architectural components (the *filters*) which communicate by dedicated one-way communications channels (the *pipes*).

Topology



- Filters
  - Self-contained, typically stateless.
  - Perform one task only.
- Pipes
  - Typically unidirectional and point-to-point.
  - Typically messages have small dimension for performance reasons.



Pipeline Architecture

- Also known as *pipes and filters,* architecture style.
- The underlying principle behind Unix Bash and Zsh.
- Functionality is decomposed in sequential steps, which will correspond to distinct architectural components (the *filters*) which communicate by dedicated one-way communications channels (the *pipes*).

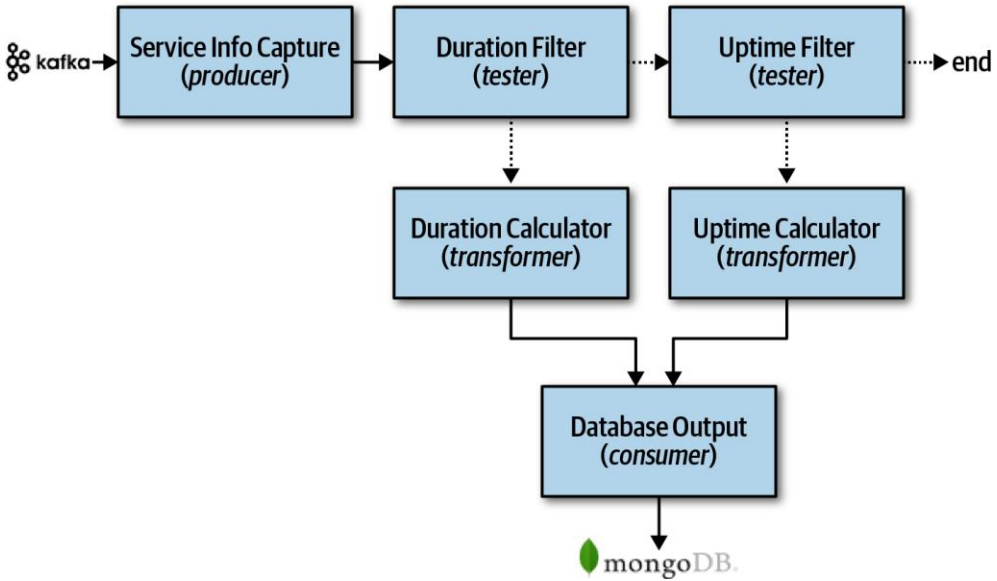


Filters

- Self-contained and usually stateless.
- There are four types of filters:
  - **Producer** – Starting point of process, outbound only, also known as *source*.
  - **Transformer** – Accepts input, optionally performs a transformation, and forwards it to outbound pipe *(map)*.
  - **Tester** – Tests one or more criteria on the input and optionally produces output based on test results *(reduce)*.
  - **Consumer** – Termination of pipeline flow. Will typically store or display the result.

Example

- **Service Info Capture** – Subscribes to Kafka and receives service information
- **Duration and Uptime Filters** – Identify and separate duration/uptime information.
- **Duration and Uptime Calculators** – Compute metrics based on raw data
- **Database Output** – Persists data in mongo DB.





Architectural Characteristics Ratings

- One architectural quanta – usually implemented as monolithic solution.
- Monolithic nature avoids complexities associated with distributed architectures.
- Simple and easy understand, they are low cost to build and maintain.
- Architectural modularity comes from separation of concerns between separated filters.
- Higher modularity improves testability and deployability with respect to layered architecture. But it is still monolithic with associated concerns.

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★★



Architectural Characteristics Ratings

- Reliability is average due to having no impact from network issues. Not great due to monolithic nature.
- Elasticity and Scalability are very low, due to monolithic deployment. Same with fault tolerance.

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★★

2.5

Microkernel Architecture Style

Frequently used for user product-based applications.

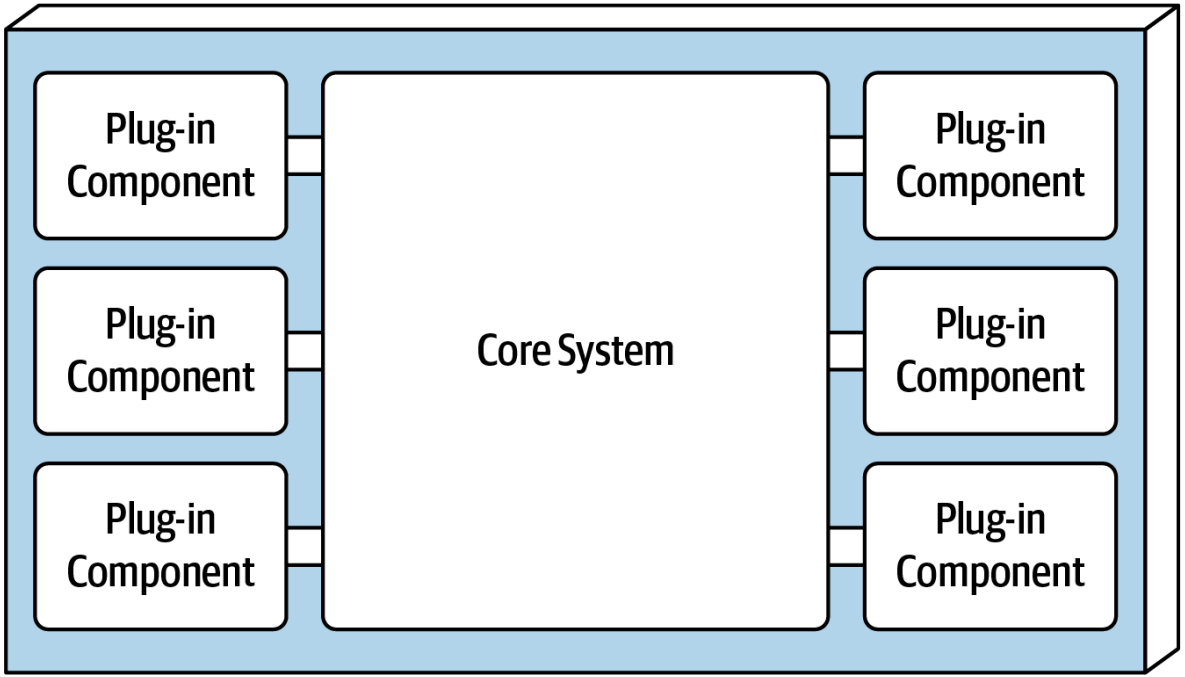


Microkernel Architecture Style

- Also known as the *plug-in* architecture style.
- Use for creating customizable products centred around a core functionality.
- It is composed by two main architectural components:
  - A core system
  - Plug-in components



Topology


Plug-in Component
Plug-in Component
Plug-in Component
Core System
Plug-in Component
Plug-in Component
Plug-in Component

Core System

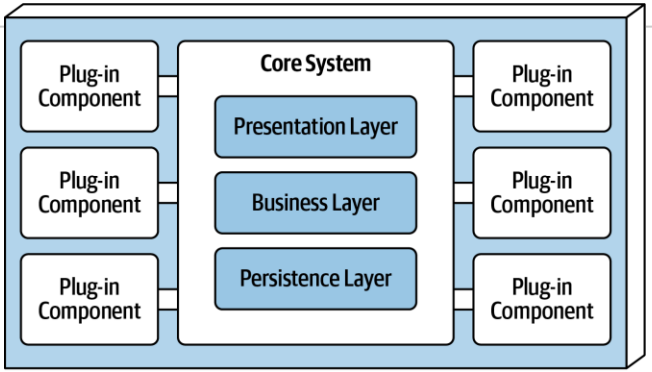
- Minimal functionality required to run the system.
- Includes only the main or “happy path” (general flow), without further customization.
- Dynamically loads plug-ins to handle custom functionality.
- E.g. Loading a plug-in to handle assessment of an external device:

```
public void assessDevice(String deviceID) {
        String plugin = pluginRegistry.get(deviceID);
        Class<?> theClass = Class.forName(plugin);
        Constructor<?> constructor = theClass.getConstructor();
        DevicePlugin devicePlugin = (DevicePlugin)constructor.newInstance();
        DevicePlugin.assess();
}
```

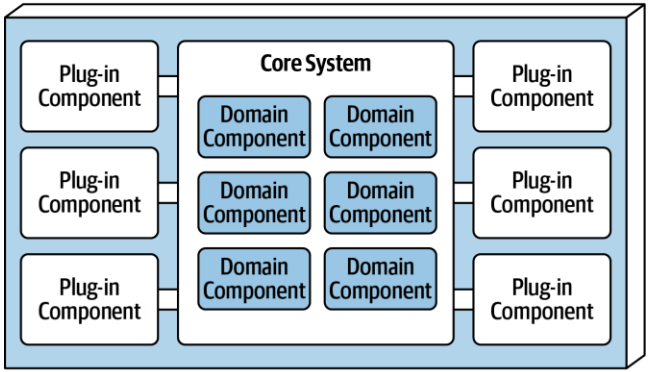


Core System

- The core system itself can be created using a technical or domain-partitioned approach



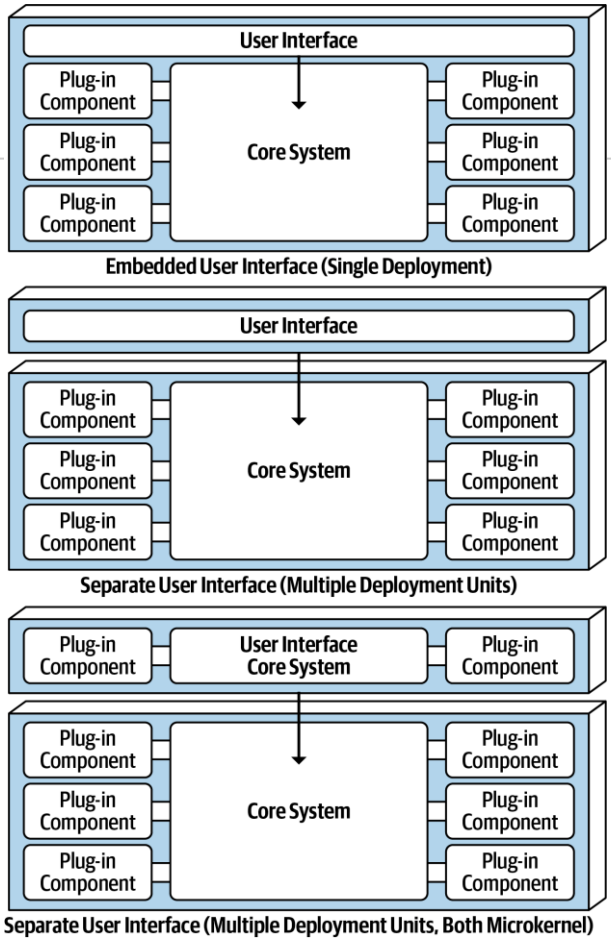
Layered Core System (Technically Partioned)



Modular Core System (Domain Partioned)

Core System

- The core system itself can be created using a technical or domain-partitioned approach.
- The UI can be embedded into the micro-kernel or can be contained in a separated deployment unit.
- The UI itself might be based on a microkernel architecture style.





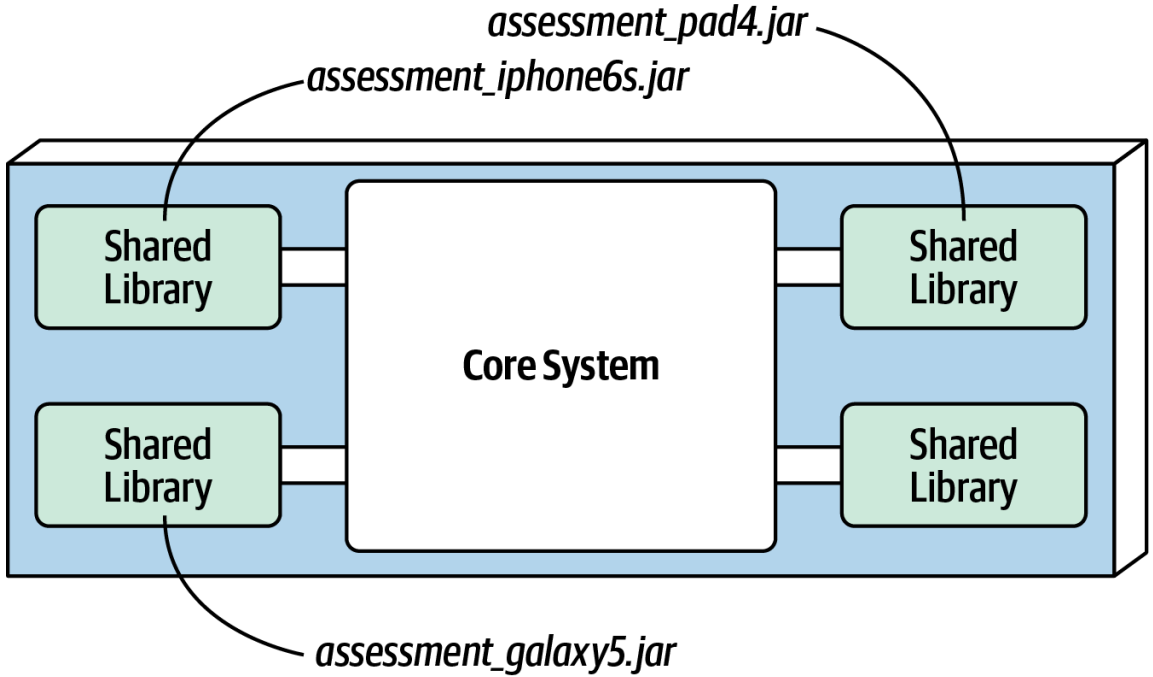
Plug-in Components

- Standalone, independent components, containing specialized processing, additional features and custom code.
  - Can contain more volatile code, isolating it from the core, leading to better testability and maintainability.
  - Communication with core is generally point-to-point (usually method invocation to an entry point).
- Runtime plug-in components can be added or removed without redeploying other components or the core system.
  - A compile-time plug-in is simple, but requires recompilation and redeployment of the entire system (core and other plugins).
- Some frameworks are designed to support runtime plug-in developments: *OSGi, Jigsaw, Prism*



Plug-in Components

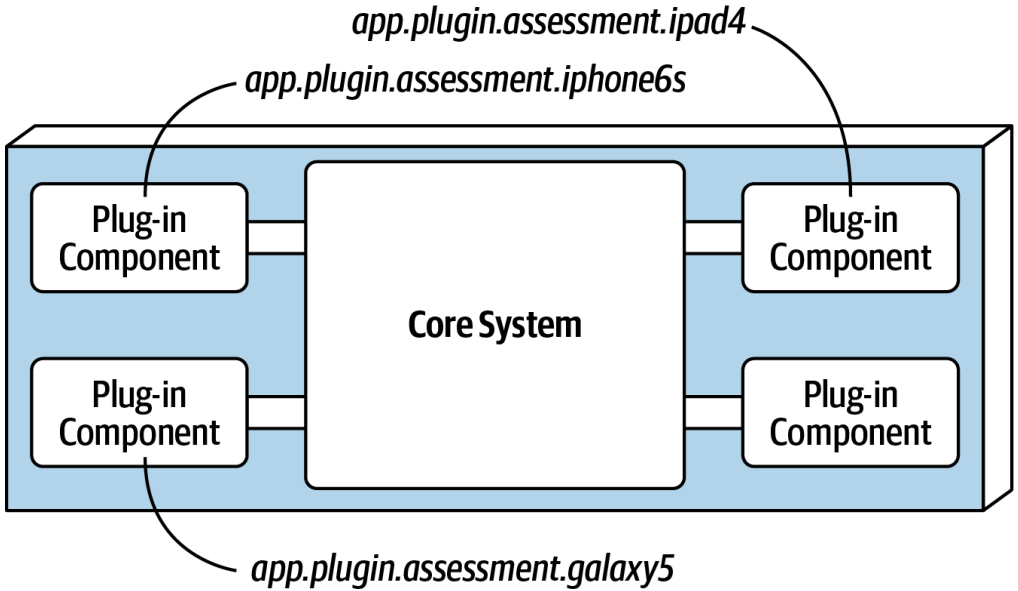
- Plug-ins can be implemented as shared libraries (jar, dll,..)





Plug-in Components

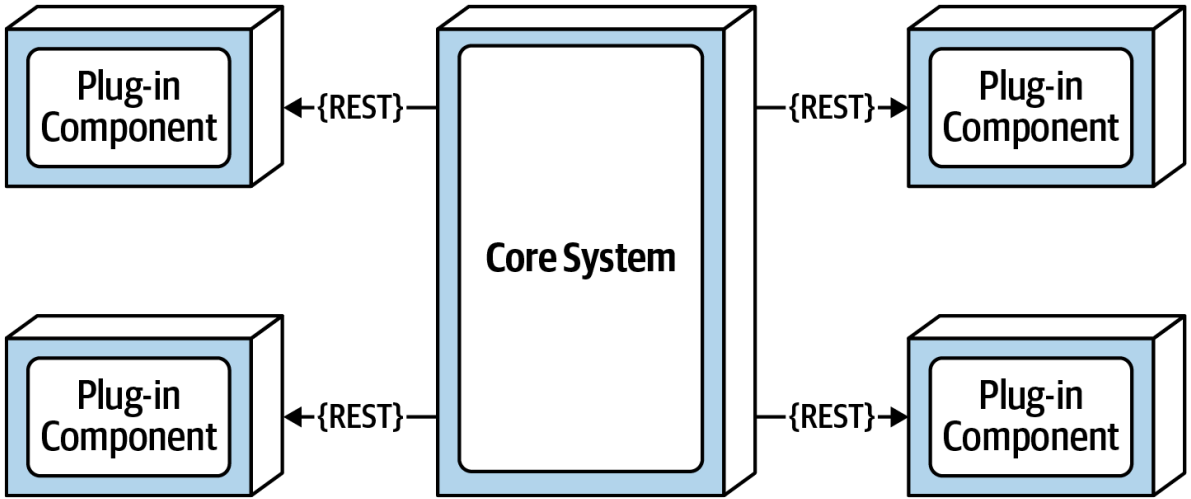
- Plug-ins can also be implemented as resources in a specific package/namespace.





Plug-in Components

- Although less common, a plug-in can also be implemented as a remote component.
  - *Still monolithic in essence due to monolithic core.*



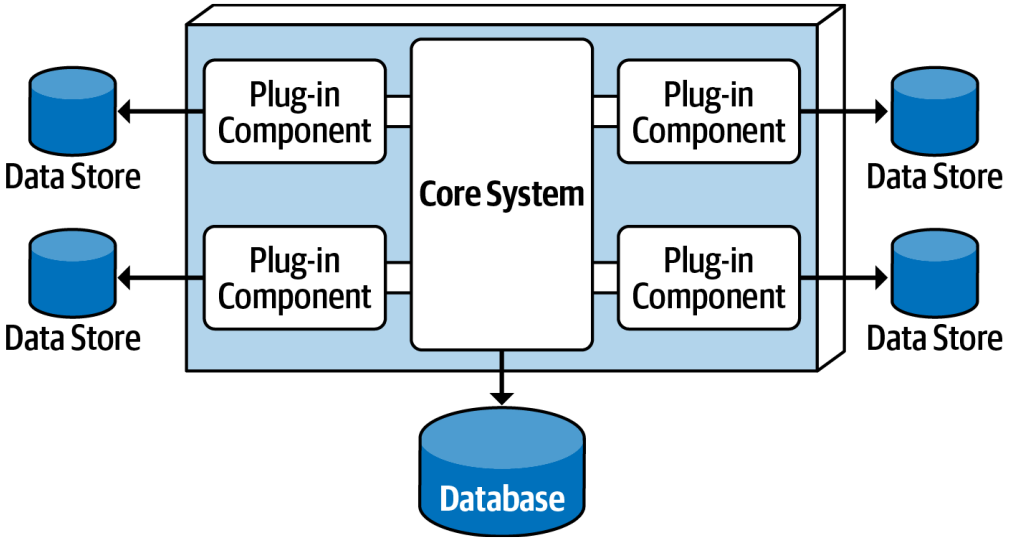


Plug-in Components

- Remote plug-ins can allow for better scalability and throughput.
- Also allow asynchronous operation.
- Negative trade-offs include added deployment complexity, networking issues, etc.

Data storage and access

- Plug-ins typically will not connect directly to a centrally shared database.
  - This is necessary for decoupling purposes.
  - Doing otherwise would significantly downgrade maintainability.
- Each plug-in may have its own localized data store.





Registry

- The core system must know what plugins are available and how to access them.
- A common approach for solving this is by using a plug-in registry. It includes information such as:
  - Plugin name
  - Data contract (input and output data, data format)
  - Remote access details (if applicable)
- The registry is usually a simple internal data structure, mapping a key to plug-in information.
  - It might be more elaborated and rely on frameworks such as Apache Zookeeper or Council, either embedded in the core or accessed externally.



Contract

- The contract between the core system and plug-ins (in a specific domain) includes the behavior, input and output data specification.
- Contracts can be implemented using JSON, XML or even binary data objects, passed back and forth between core and plug-in.
- Example: In the device assessment example, the plugins might be required to implement a specific interface and return a certain data structured.

```
public interface AssessmentPlugin {
public AssessmentOutput assess();
public String register();
public String deregister();
}
public class AssessmentOutput {
public String assessmentReport;
public Boolean resell;
public Double value;
public Double resellPrice;
}
```



Examples and use cases

- Software development tools:
  - Eclipse IDE
  - PMD
  - Jira
  - Wildfly
- Internet Browsers:
  - Chrome
  - Firefox
- Applications in highly regulated domains:
  - Insurance
  - Fiscal/Legal



Architectural Characteristics Ratings

- Shares similar traits to other monolithic solutions:
  - One quanta (all requests go through core)
  - Simplicity and low cost
  - Low elasticity, scalability and fault tolerance.
- Can be both domain or technically partitioned.
- Testability, deployability and reliability are improved because functionality is contained in independent plug-ins.
- Modularity and extensibility are reasonable, functionality can be added, removed or changed through independent plug-ins.
- Performance tends to be better than layered due to being less prone to the architectural sinkhole pattern.
- Typically smaller in size, because functionality is composed.

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★★
Reliability	★★★
Scalability	★
Simplicity	★★★★
Testability	★★★

2.6

Service-Based Architecture Style

One of the most simpler and more pragmatic distributed architectural styles

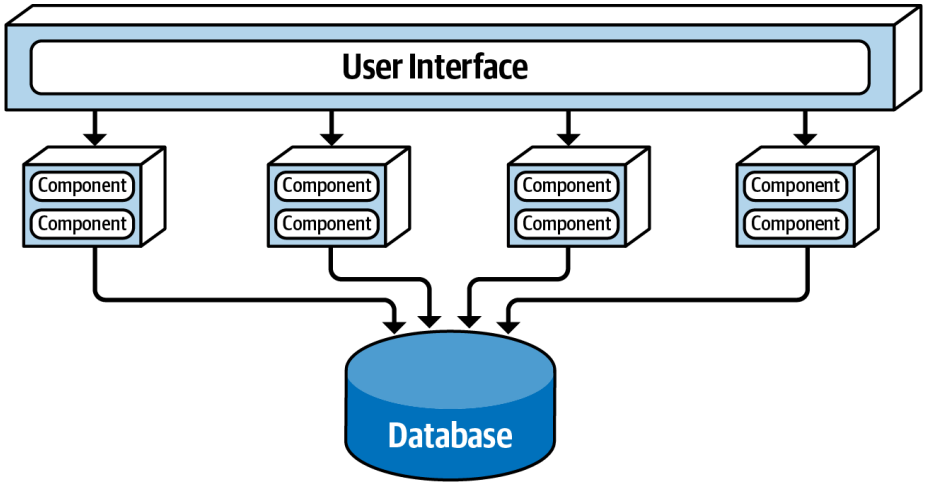


Service-Based Architecture Style

- A simple, architecturally flexible approach.
- Hybrid of the microservices architectural style
- Lower level of complexity than event-driven architecture or microservices architecture.
- Popular choice for business applications.

Topology

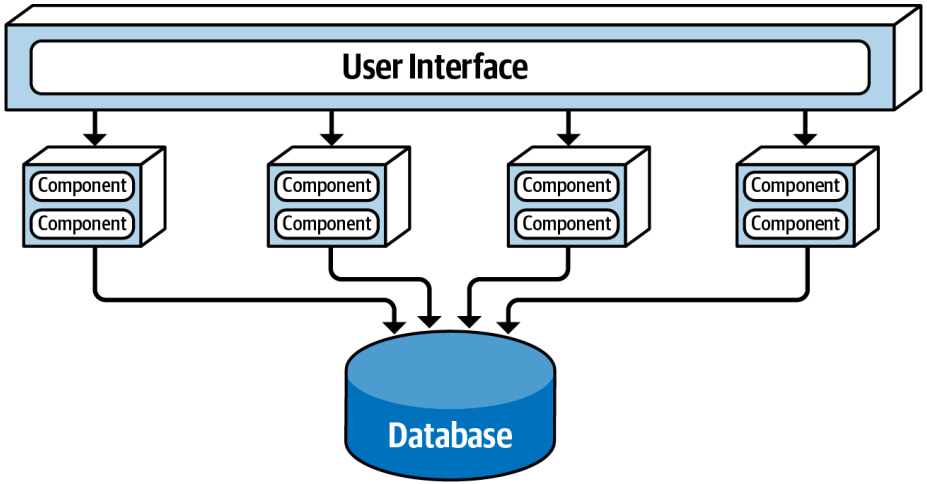
- Distributed macro layered structure with separately deployed user interface.
- Services are coarse-grained “portions of the application” (domain services) independently and separately deployed.
- Services can be, but are not necessarily, containerized (e.g. using a Docker image).
- A single monolithic database is usually shared, limiting the number of domain services (4-12 would be typical)





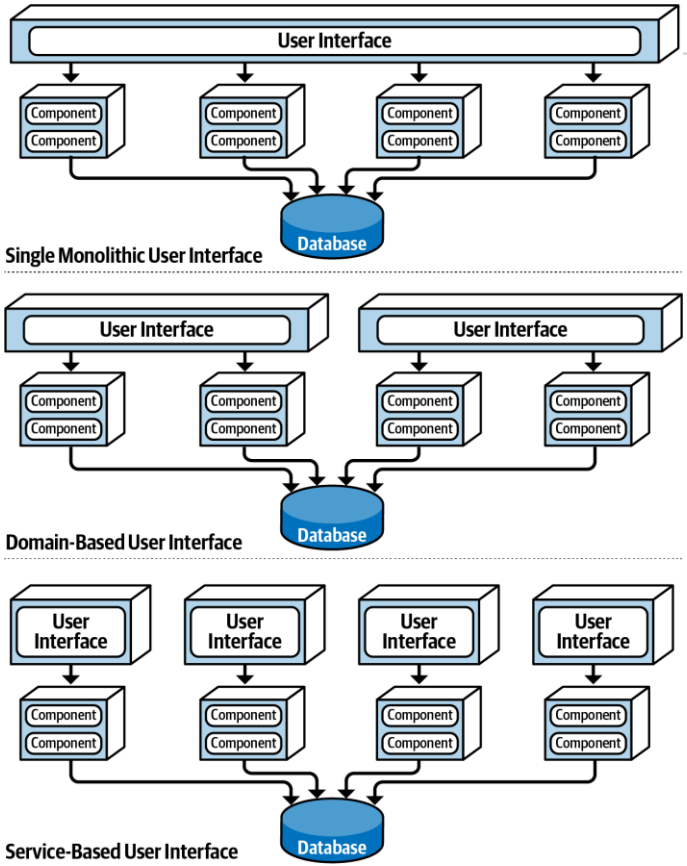
Topology

- Usually, only a single instance of a domain service exists.
  - But multiple instances can be created for improving scalability, fault tolerance, and throughput.
  - A load-balancing intermediate is then used between the user interface and replicated services.
- A remote access protocol is used to communicate from user interface to services.
  - REST, RPC, SOAP



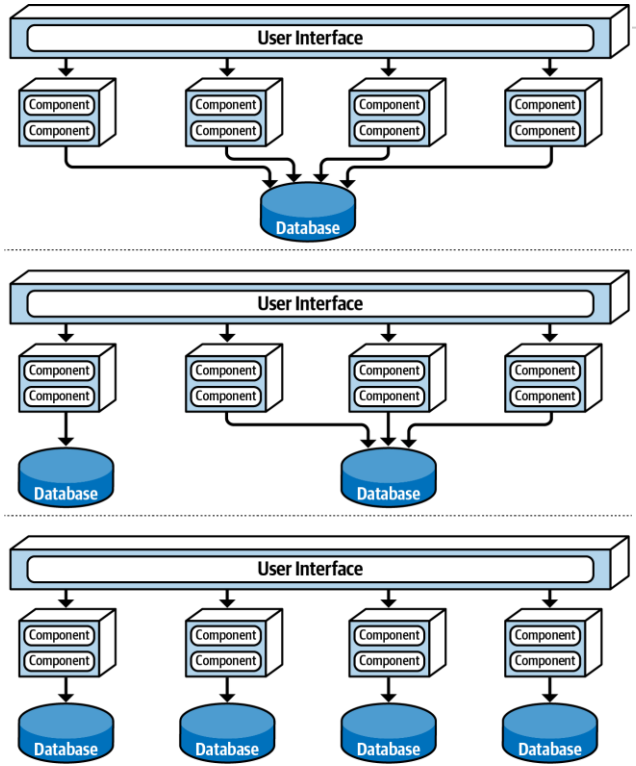
Topology Variants

- Many variations can be created, highlighting the flexibility of this approach.
- Variations on the user interface side are shown on the image.



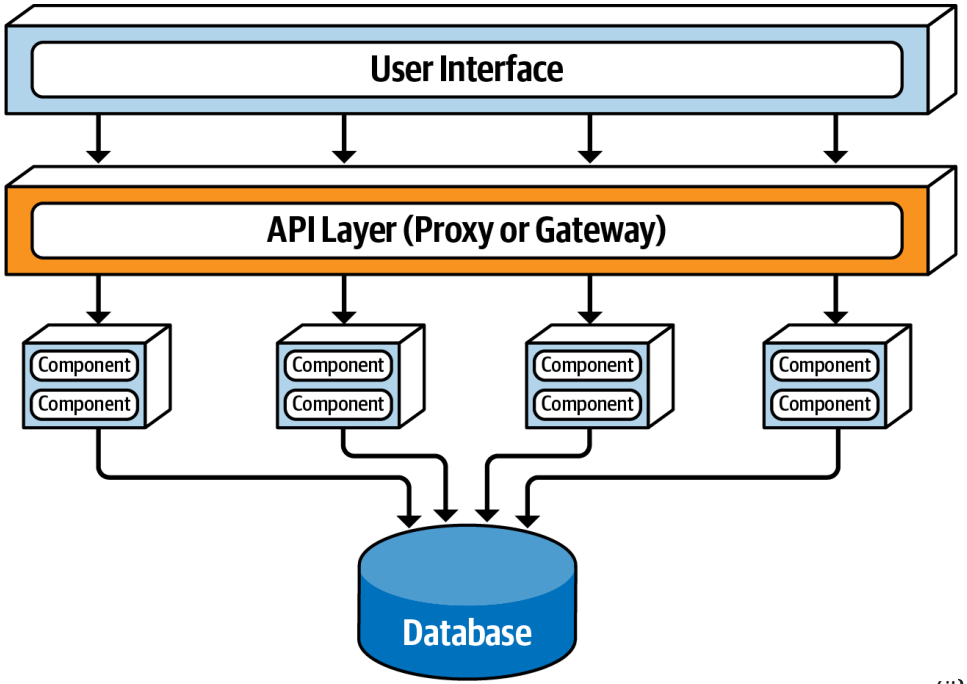
Topology Variants

- Many variations can be created, highlighting the flexibility of this approach.
- Variations on database deployment are also possible.
- Inter-service communication is not desirable, so all services have direct access and operate on independent data (also avoiding duplications).



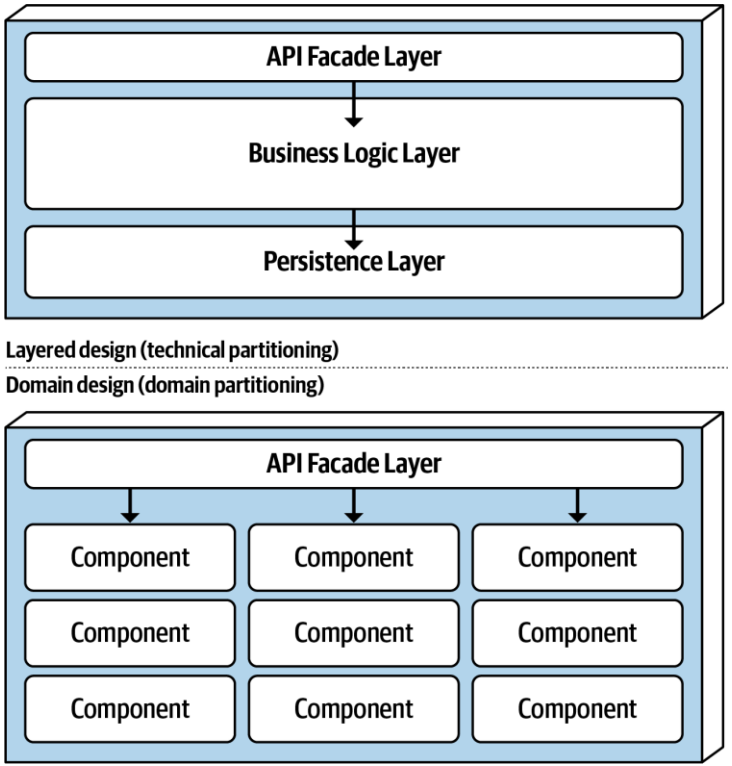
Topology Variants

- Many variations can be created, highlighting the flexibility of this approach.
- An API Layer can also be added to isolate services from clients.
  - This layer can implement crosscutting concerns such as metrics, security, and service discovery.



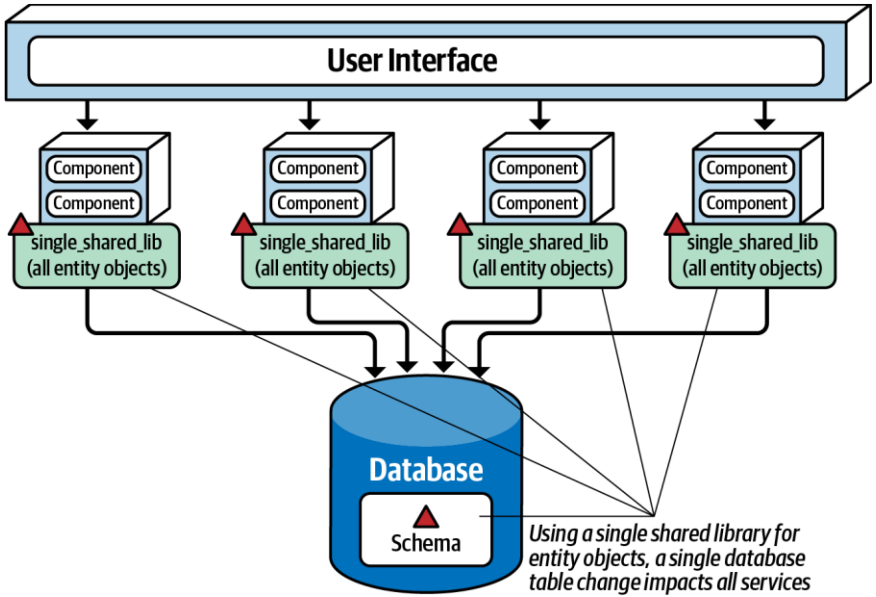
Service Design and Granularity

- Each service is coarse grained, and is typically designed using a layered approach, using an API facade layer, business layer and persistence layer.
- Another approach is to create a partition of the domain of each domain service, resulting in a modular monolith.
- The API façade performs internal orchestration of the requests.
  - In microservices architectures, orchestration is usually external. This is an important difference.
- ACID properties are maintained within each domain service.



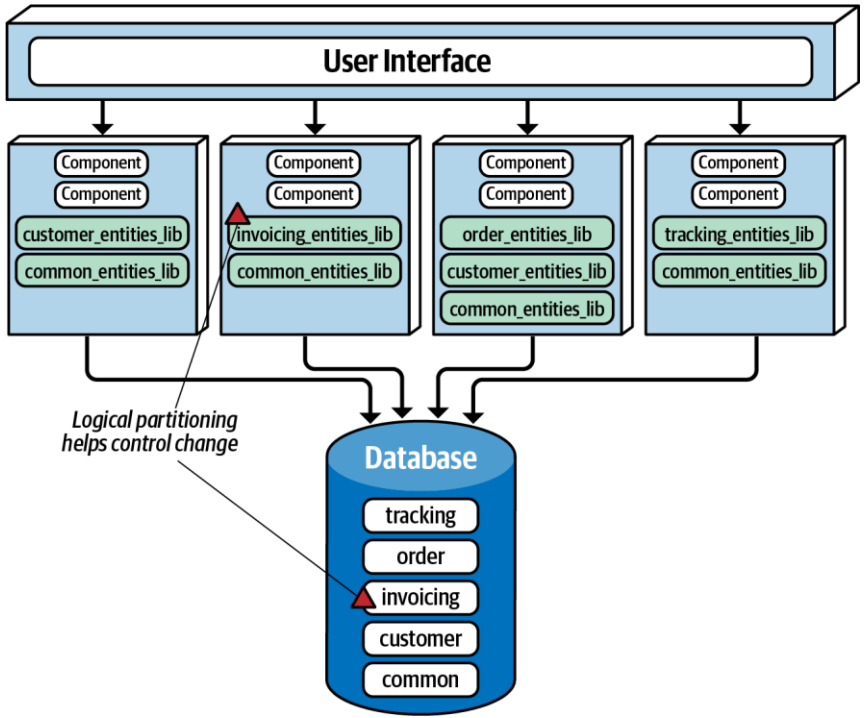
Shared Database

- When a monolithic database is shared between all services, a custom shared entity library can be shared by all services.
- This make all services vulnerable to schema changes.



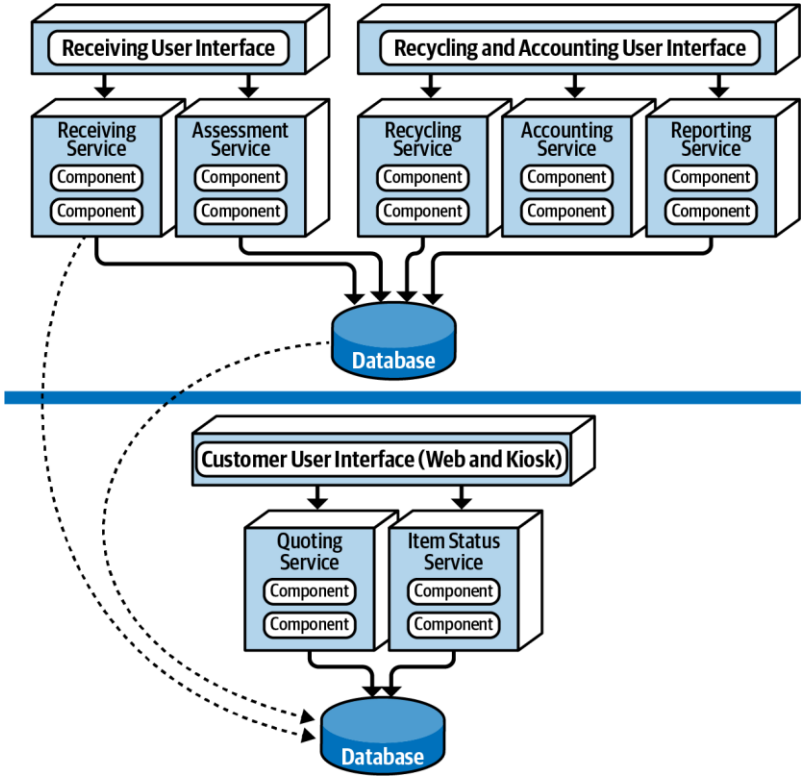
Shared Database

- When a monolithic database is shared between all services, a custom shared entity library can be shared by all services.
- This make all services vulnerable to schema changes.
- An alternative to mitigate this is to partition entities into finer-grained libraries specific to each service.





Example: Electronics Recycling System





Architectural Characteristics Ratings

- Performs very well in all categories.
- The coarse-grained nature of services negatively impacts elasticity and scalability.
- Simplicity and cost are reasonable, when comparing with other distributed architectures.
- Reliability is high due to the coarse grained nature of the services (less reliant on network services).
- Breaking into multiple separatedly deployed domain services allows faster change (agility), better test coverage due to limited scope (testability) and allows more frequent deployments with less risk (deployability).

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★



Architectural Styles

- To be continued next class....